

Building Prompt Kit: A Design System for AI-Driven Design

Reduced landing page design and revision time from 2 weeks to 2–4 days, an approximately 85% reduction, by building an AI-readable design system that let Claude Code generate on-brand, production-ready Figma files.

ROLE — Sr. UX/UI Designer

YEAR — 2026

DURATION — Design system MVP built in 2–3 weeks

DELIVERABLES — Design system architecture, component library (Foundations + Components), AI-readability standards, naming conventions, design-to-agent handoff documentation

HOW PROMPT KIT IS STRUCTURED

Inside the system

Prompt Kit lives in a single Figma file, organized into Foundations and Components pages, with a reserved Documentation section for future usage guidelines. I originally split these into three separate files, but quickly learned that consolidating into one made the system much easier to maintain and far more reliable for AI agents to reference consistently.

Foundations — primitive colour tokens (33 variables across 9 semantic groups), typography variables, spacing, and radius tokens. This is the source of truth every component pulls from.

Components — started as a mapped set of 8 core components, and grew to around 50 as new patterns, verticals, and edge cases demanded it. Each one is built with AI-readable naming (PascalCase components, semantic variants like State=focused instead of something like Variant2).

I also planned a **Templates** section early on, but abandoned it once I realized static pages couldn't keep up with how often our needs shifted by brand and vertical.

[Space for screenshot: Prompt Kit file page structure in Figma]

DISCOVERY

The realization that started it all

Towards the end of April, I tested a handful of AI design tools, Lovable, Magic Path, and Figma Make, to see if we could rebuild our landing pages by prompting instead of designing by hand. The results were inconsistent. Without a real design system to reference, these tools defaulted to their own generic pattern libraries instead of matching our brand.

That was the moment it clicked: if we wanted AI to design for us, it needed something structured to design from. Prompting alone wasn't going to get us there.

REALITY CHECK

3 constraints I ran into building the system

01

Figma's variable bugs

Text styles silently disconnected from their linked variables, so the numbers shown didn't always reflect what was actually applied.

02

Mobile vs. desktop typography

Figma applied mobile type styles inconsistently across breakpoints, forcing a full redesign of how type was structured.

03

Speed vs. architecture

Building fully tokenized components from scratch was the right long-term call, but it was slow, every component surfaced a new Figma quirk to debug.

DIRECTION

The build decision: bottom-up, top-down, or hybrid

Rather than default to one approach, I weighed three ways to build the system.

- **Bottom-up** — build every token and component from scratch, fully architected from day one. Slower, but nothing inherited hidden problems.
- **Top-down** — take our existing Unbounce-derived Figma designs and componentize what already worked. Fast, but risked baking in hardcoded values and inconsistent structure.
- **Hybrid** — rebuild the components where architecture mattered most, and reuse existing assets where the visual was already right.

I chose hybrid. Atomic pieces like the Button and Form Input needed proper variant properties to be reliably usable by AI, so those were built from scratch. Icons and larger compositions like the Form Card were faster to adapt from existing designs, since their visual fidelity was already proven.

The test that mattered: AI agents don't care how a system was built, they care whether it's consistent. So the real question wasn't which approach was "more correct," it was which one produced tokens, naming, and **component architecture an AI agent could actually rely on.**

ACTIVITIES & OUTPUTS

What actually got built

By the end of the first sprint, Prompt Kit had grown from an idea into a working, documented system.

Foundations

- 33 colour primitives across 9 semantic groups
- 25 typography variables with desktop and mobile breakpoints
- 8 spacing tokens, 5 radius tokens, 6 effect styles
- 10 published text styles

Components

- Started with 8 core components, grew to roughly 50 as new patterns and verticals demanded it
- Built with AI-readable naming conventions from day one, PascalCase components, semantic variant names

Key architectural decisions

- Chose a lean system over an exhaustive one, only building what had a real use case
- Standardized variant properties (Size, Type, State) across every component
- Built in Auto Layout discipline throughout, critical for AI agents to resize and rearrange components reliably

A LIVING SYSTEM

The system kept growing, on purpose

Prompt Kit was never meant to be finished on day one. Every new landing page surfaced needs the original 8 components didn't cover, a new brand's visual treatment, a vertical-specific layout, a fresh way to showcase a section. Rather than force those needs into existing components, I added new ones.

By day 46, that process had grown the library from 8 components to roughly 80. The growth wasn't scope creep, **it was the system doing exactly what it was designed to do: absorb**

real production demands and get more capable with every page built, instead of staying a fixed, one-time deliverable.

LESSON LEARNED

Size components by their content, not a fixed frame

Instead of forcing components into fixed dimensions, I sized them by their content and padding tokens. **A button that sizes itself around its own padding stays consistent no matter where it's dropped**, while a button forced into a fixed size breaks the moment its content changes. This one decision is what let dozens of landing pages share the same components without constant one-off adjustments.

The mistake I caught late

Early on, I let Figma's default convention handle responsiveness, one typography style per type, with Figma automatically swapping in a predetermined mobile size whenever a mobile frame was used. It seemed efficient, but it meant I never had direct control over how each style behaved across breakpoints.

The fix was to stop relying on that automatic swap. I built two separate typography variables for every style, one for desktop and one for mobile, and named them distinctly so there was no ambiguity about which was which. It took more upfront setup, but it meant every type style behaved exactly as intended on every screen, instead of hoping Figma's default logic got it right.

UNEXPECTED SHIFT

When a layout fix becomes an architecture decision

Matching the original 304px mobile header exactly caused my tokenized button to overflow its space. The real problem wasn't the button, it was that I hadn't decided how viewport width and content width should relate to each other.

The fix was a simple pattern: a 390px outer frame holding a 304px inner frame for the logo and button. Separating viewport size from content size solved the overflow, and the same pattern became reusable for the Hero section too.

What felt like a small layout fix was actually a system decision I hadn't made yet, and making it properly is what kept the system AI-readable.

THE PIVOT

A platform shift I didn't see coming

Midway through the sprint, Google feedback led the company to a discovery: search pages built on WordPress tended to rank better than those built on Unbounce. That meant a real shift, search landing pages would now be built on WordPress by an outside agency, not Unbounce as originally planned.

That put a real decision in front of me. Did Prompt Kit need to change to be WordPress-friendly, or did I need an entirely separate design system for a separate platform? Getting this wrong meant either a system that only worked for half our pages, or duplicate work maintaining two systems in parallel.

VALIDATION

Someone else builds it the same way

I met with our agency's lead developer, who was handling the new WordPress builds, to work through exactly that question. Without prompting, he described their own approach to design systems, and it matched Prompt Kit's structure almost exactly. The only addition he asked for in the handoff was clearly defined design tokens for each landing page, something Prompt Kit was already built to provide.

That resolved the dilemma. Prompt Kit didn't need to be rebuilt or split in two, it needed to travel with clearly defined tokens, which is exactly the discipline it was built around from day one. Unprompted agreement from a team with no reason to flatter the work is hard to manufacture, and it confirmed the system's architecture held up outside the tool it was designed for.

REFLECTION

What I'd carry into the next system

Prompt Kit was actually one piece of a larger 75-day agentic design sprint led by our director of CRO, aimed at designing faster, higher-quality, better-converting pages. Building the design system was the foundation, step one of that plan, and by the time it was in active production use, the real work shifted from building the system to using it: pairing it with AI agents to actually generate pages, which became its own project entirely.

Before that shift, the remaining work on Prompt Kit itself was maintenance and discipline: unpublishing legacy libraries that had been superseded, keeping a change log for major token and component updates, and setting up a quarterly audit cadence so drift wouldn't quietly build back up over time.

A few things I'd carry forward into any future design system:

- **Build lean, not exhaustive.** Starting with 8 components instead of trying to anticipate every need meant the system grew in response to real demand, not speculation.
- **AI-readability and human-readability turned out to be the same discipline.** Clear naming, tokenized values, and consistent structure made the system easier for people to use too, not just agents.
- **A design system is a living product, not a one-time deliverable.** It needs the same ongoing care as anything else in production, which is exactly what the post-sprint backlog reflected.